SOFTWARE DEVELOPMENT WITH PYTHON

Creating and Defining Database Structures

Tables, Data Types, Identifiers, and Structural Change

Module 2 of 3 · Day 2: SQL Fundamentals and Relational Database Design
Running project: **Student Course Enrollment API** · Target: **PostgreSQL 18**

Prepared 19 October 2026 · psql and VS Code · Built on Module 1

Contents

What this module is, and what it is not 2

The one idea this module rests on 2

1 Project folder setup 2

1.1 The folder tree . . . 3
1.2 Which file gets which code . . . 3
1.3 `.gitignore` . . . 4
1.4 `.env.example` . . . 4
1.5 `README.md` . . . 5

2 Before you begin 6

3 Creating your first table 7

3.1 Naming rules and the lowercase surprise . . . 8

4 Choosing data types 8

4.1 The text-everything antipattern . . . 8
4.2 Text: `text`, `varchar(n)`, `char(n)` . . . 9
4.3 Whole numbers: `smallint`, `integer`, `bigint` . . . 9
4.4 Exact decimals: `numeric` . . . 9
4.5 Dates and times: `date`, `timestamp`, `timestamptz` . . . 10
4.6 Booleans . . . 10
4.7 The decision table . . . 11
4.8 Two design habits, named explicitly . . . 11
4.9 Building the second table . . . 11

5 Auto-generated identifiers and UUIDs 12

5.1 The problem first . . . 12
5.2 The old spelling and the standard one . . . 12
5.3 Where the counter went . . . 13
5.4 UUIDs — the alternative identifier . . . 13

6 Changing a table: `ALTER TABLE` 14

7 Removing a table: `DROP TABLE`, and the safety net 15

8 The script file, and proving it is re-runnable **17**

8.1 Running it — three routes, one file 19

8.2 Prove it is re-runnable 19

9 Verification checklist **20**

10 Troubleshooting **20**

11 Instructor notes **21**

12 Module 2 submission **22**

What this module is, and what it is not

This module teaches students to **build the containers that will hold their data**: creating tables with `CREATE TABLE`, choosing the right data type for every column, letting PostgreSQL generate identifiers automatically, changing structure safely with `ALTER TABLE`, and removing tables with `DROP TABLE`.

It is **not** about the data itself. Nothing is inserted, read, updated or deleted in this module — that is CRUD, and it is Module 4. The full constraints treatment (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`) is Module 3; here we use only `NOT NULL` and `DEFAULT`, which are part of basic column definition.

The `enrollments` table, the third of the running project, is deliberately **not** built here, because it requires relationships between tables. Students should finish this module feeling that gap. It is the hook into the next session, and it should not be papered over.

One change from your earlier notes

> If you drafted this schema yourself before reading further: notice that `credits` and `course_fee` carry **no `DEFAULT`**. A default of `1` or `0` would quietly invent data the registrar never supplied. The class rule this module teaches is that **the only value the server should invent for you is the timestamp**.

The one idea this module rests on

The one idea this module rests on

> **A table definition is a contract, and the server enforces it.**

In the students' Python work, a record was a dictionary: `student = {"name": ..., "age": ...}`. Nothing stopped anyone putting `"age": "twenty"` into it, or omitting a key entirely. A dictionary is a suggestion. A table definition is a promise with enforcement behind it.

When a column is declared `date`, the server rejects anything that is not a real date — not at review time, not at testing time, at *write* time, every time, forever. When a column is marked `NOT NULL`, no row can exist without it. When identifier generation is handed to the server, no two rows can collide no matter how many API processes write simultaneously.

Every data bug these students will ever debug was either prevented or caused by the schema. This module is where that is decided.

Project folder setup

Everything for this course lives in one project folder, created once. Students create it in VS Code: **File > Open Folder**, name it `student-enrollment-api`, and place it wherever they keep their course work.

The folder tree

Listing 1. The project folder, as it exists at the end of this module

```
student-enrollment-api/├──

 README.md                    ← what the project is + current status├──
 .gitignore                   ← files that must never reach a repository├──
 .env.example                 ← template for the connection URL├──

 db/│
   ├── sql/                     ← ALL SQL lives here, numbered in order│
   │   └── 01_create_tables.sql  ← ★ this module's code goes here│
   └── notes.md                 ← their own words, their own folder├──

 app/                         ← Python code — later stages, not today├──
 tests/                       ← later stages└──
 .vscode/                     ← VS Code creates this itself
```

The root folder uses hyphens, `student-enrollment-api`, which is fine because a project folder is never imported by Python. The Python package inside it will be `app`, lowercase and underscored — that is the rule which actually matters later.

Which file gets which code

File	What goes in it	When
`README.md`	Project name, description, how to run the SQL, a checklist of progress	now
`.gitignore`	`.env`, Python caches, `.vscode/settings.json`	now
`.env.example`	Placeholder connection string, no real password	now
`db/sql/01_create_tables.sql`	**This module's `CREATE TABLE` statements**	now
`db/notes.md`	Free notes — types chosen, and why; open questions	now
`app/, tests/`	The FastAPI application and its tests	later stages

Two entries in the files that follow are deliberate. `.env` will hold the real database password. `.vscode/settings.json` holds whatever connection settings a student decided to type there, which is a direct payoff from the Module 1 warning about database passwords stored in plain text. Putting both in `.gitignore` turns a habit students must remember into a rule the project enforces.

.gitignore

Listing 2. The .gitignore file

```
# --- Secrets: never commit these ---
.env

# --- Python artefacts (from later modules) ---
__pycache__/
*.py[cod]
.venv/
venv/

# --- VS Code local settings: may contain credentials ---
.vscode/settings.json

# --- Database dumps: large and machine-specific ---
*.dump
*.sql.gz
```

Line by line

Line	Why it is there
`# --- ... ---`	A comment. Git ignores everything after `#`, so the file can explain itself six months from now.
`.env`	Ignores the file holding the real database URL and password. A pushed password is a leaked password, even a purely local one.
`__pycache__/`	Ignores Python's compiled bytecode cache. Machine-specific junk, regenerated automatically.
`*.py[cod]`	Ignores `.pyc`, `.pyo` and `.pyd` files. The bracket form catches all three with one line.
`.venv/, venv/`	Ignores the virtual environment folder. Thousands of files, never shared, rebuilt with one command.
`.vscode/settings.json`	Ignores *that one file* — not the whole `.vscode` folder. Teams legitimately share `extensions.json`; `settings.json` is the one that can leak credentials.
`*.dump, *.sql.gz`	Ignores database backup files. Large, personal, never part of source code.

.env.example

Nothing in this course reads `.env` yet — it arrives with the API modules. It is created now so the connection string is written down in one agreed format before it is needed in a hurry.

Listing 3. The .env.example template

```
# Copy this file to ".env" and put your real password in the copy.
# The copy is gitignored. This template is not.
# Nothing in this course reads .env yet - it arrives with the API modules.

DATABASE_URL=postgresql://student:CHANGE_ME@localhost:5432/school_db
```

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=school_db
DB_USER=student
DB_PASSWORD=CHANGE_ME
```

Line by line

Line	Why it is there
`The three # comments`	Instructions for whoever opens the file. `CHANGE_ME` is a loud marker — it cannot be shipped by accident.
`DATABASE_URL=`	One connection string in URL form, `postgresql://user:password@host:port/database` — the same shape used in the Module 1 VS Code extension.
`The five DB_* lines`	The same five values broken out separately. Some tools and some examples want them individually. Both forms are normal; the codebase settles on one.

README.md

Students fill this in as the course progresses. It is the first thing anyone reading their repository will see, and it doubles as a study checklist.

Listing 4. The README.md template

```
# Student Enrollment API

A small API for managing students, the courses they offer, and the
enrollments that connect the two. Built in stages.

## Progress
- [x] Day 1, Module 4 - PostgreSQL installed and connected (psql + VS Code)
- [x] Day 2, Module 2 - students and courses tables created
- [ ] Day 2, Module 3 - constraints and keys, and the enrollments table
- [ ] Day 2, Module 4 - CRUD

## Running the SQL
From the project folder:

    psql -U student -d school_db -f db/sql/01_create_tables.sql

Then check what you built:

    psql -U student -d school_db
    school_db=# \dt
    school_db=# \d students
```

Line by line

Line	Why it is there
`# Student Enrollment API`	A Markdown level-one heading; the `#` makes it the largest heading on the page.
`The description paragraph`	States what the project *is*, in plain words.
`## Progress`	A second-level heading introducing the task list.
`- [x] and - [ ]`	A Markdown task list. `[x]` renders as a completed checkbox.
`## Running the SQL`	The heading separating the project description from its instructions.
`The indented command lines`	A fenced code block, which preserves spacing and stops Markdown from reformatting the command.

Before you begin

Students connect as their `student` role to `school_db`, then confirm two things about where they are standing.

Listing 5. Confirming the connection and the starting state

```
\conninfo
\dt
```

Line by line

Line	What it does, and why
`\conninfo`	A **psql command**. It prints the current connection: host, port, database, user. The single most useful command when something behaves unexpectedly, because it answers *am I even in the right database?* before any debugging begins.
`\dt`	Lists the **tables** in the current database. `\dt+` adds size and description. It should report **Did not find any relation**.

“Did not find any relation” is a success message

Students routinely read this as a failure and try to fix it. It is the correct empty state: the database exists, the connection works, and the schema is simply empty — which is exactly what they are about to change. Say so explicitly, because a student who believes they are broken will not press on.

Reinforce what Module 1 established. Backslash commands are handled by `psql` itself and take **no semicolon**. `CREATE TABLE` is SQL: it travels to the server and **must** end with `;`.

The semicolon trap

If students press Enter and `psql` answers with a new prompt that reads `school_db-#` — a **dash** where the equals sign usually is — nothing has run. `psql` is waiting for the statement to finish. Type `;` and press Enter.

> Half of all beginner reports that “PostgreSQL has frozen” are an unfinished statement, and the prompt itself is the only evidence. Teach students to read the prompt character before reaching for the power button.

Creating your first table

The students’ project needs three tables. This module builds the first two. Begin with the statement in full, then take it apart.

Listing 6. Creating the students table

```
CREATE TABLE students (
    id              integer GENERATED BY DEFAULT AS IDENTITY,
    full_name       text NOT NULL,
    email           text NOT NULL,
    date_of_birth   date,
    created_at      timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);
```

Line by line

Line	What it does, and why
`CREATE TABLE students (`	Opens a new table named `students`. Everything between the parentheses is its structure.
`id integer`	An integer column. The rest of the line makes the server fill it in. Nobody types an id; it cannot be forgotten, and it cannot collide when two requests arrive together.
`full_name text NOT NULL,`	Any-length text, always required. `NOT NULL` makes “must be provided” a rule the server enforces rather than a rule students remember.
`email text NOT NULL,`	Any-length text, always required. Uniqueness of email is a real requirement — but it is a *constraint*, and constraints are Module 3. Noted now, added next session.
`date_of_birth date,`	A calendar date, with no time and no time zone, and no `NOT NULL`: not every student will have given a date of birth, and nullable means that absence is representable.
`created_at timestamptz`	A timestamp carrying a time zone. Two decisions at once — the type, and the default on the next line.
`DEFAULT now(),`	Fills the column with the server’s current instant at the moment the row is created.
`PRIMARY KEY (id)`	Declares `id` the primary key. Every table in this course gets one. It guarantees each row is uniquely identifiable, which is what every API lookup will later depend on.
`);`	Closes the parenthesis and terminates the statement. The semicolon is not optional — see the trap above.

Run it. `psql` answers with a single line:

Listing 7. The server’s confirmation

```
CREATE TABLE
```

That is a **command tag** — `psql`'s way of reporting what the server did. Now verify, exactly as Module 1 taught:

Listing 8. Confirming the table exists and reading its structure

```
\dt
\d students
```

Line by line

Line	What it does, and why
`\dt`	Lists tables. `students` now appears.
`\d students`	**Describes** the table: every column, its type, whether it may be null, its default, and the primary key with the unique index that a key automatically creates.

Make `\d` a reflex

`\d` is the closest thing SQL has to `print()`. From this module onward, every structural change is confirmed with `\d` before it is believed. A student who runs `\d students` after every change will never spend an hour wondering whether the change worked.

Naming rules and the lowercase surprise

PostgreSQL folds unquoted identifiers to lowercase. Type `CREATE TABLE TestTable (...)` and `\dt` will show `testtable`. Names may contain letters, digits and underscores, and may not begin with a digit.

The class convention, which matches both the wider SQL world and the students' own FastAPI work:

- **Tables**: plural, lowercase, snake_case — `students`, `courses`, `enrollments`.
- **Columns**: singular, lowercase, snake_case — `full_name`, `date_of_birth`.
- **Never quote identifiers.** Quoted mixed-case names are a trap that makes every future query more difficult to write and easier to get wrong.

Choosing data types

This is the heart of the module and deserves the most teaching time. Open with the failure that data types exist to prevent.

The text-everything antipattern

Suppose every column were text. Dates sort alphabetically: `'2025-03-09'` sorts before `'2025-10-01'` by luck, but `'9/3/2025'` sorts *after* `'10/3/2025'` — wrong. Ages

become un-addable. The value `true` arrives as `True`, `TRUE`, `1` and `yes`, written by four different people on four different days. Nothing is enforced; everything is convention.

The type system is the fix. A declared type is checked by the server on *every* write, it determines how values sort and compare, and it tells every future developer — and every library that touches this database later — what the column means.

Text: `text`, `varchar(n)`, `char(n)`

In PostgreSQL, `text` and `varchar` are **identical in storage and performance**. This surprises anyone who learned an older database engine where one was measurably faster; in PostgreSQL the two share an implementation, and a student who has been told otherwise should be told again.

`varchar(120)` is `text` plus a length check that rejects longer values. Use it only when a length limit is a genuine business rule, not decoration. `char(n)` pads every value with spaces out to the full length — `'NG'::char(5)` is stored as `'NG   '` — and is almost never what anyone wants.

Class rule

> **Default to `text`.** Names, emails, titles and codes are all `text` in this project. Reach for `varchar(n)` only when you can name the business rule that enforces the limit.

Whole numbers: `smallint`, `integer`, `bigint`

All three are exact. Only the range differs. `smallint` reaches roughly ±32,000. `integer` reaches roughly ±2.1 billion, far beyond anything this course will build. `bigint` reaches roughly ±9.2 quintillion.

Class rule

> **Use `integer` unless you can name a reason otherwise.** A course's `credits` is an `integer`.

Exact decimals: `numeric`

Run this live, slowly. It is the most durable two minutes in SQL teaching.

Listing 9. Why money is never a float

```
SELECT 0.1::float8 + 0.2::float8;
SELECT 0.1::numeric + 0.2::numeric;
```

Line by line

Line	What it does, and why
`0.1::float8`	`::` is PostgreSQL's cast operator; `float8` is the eight-byte floating-point type, also written `double precision`.
`+ 0.2::float8;`	Adds two approximations and returns `0.30000000000000004`. Binary floating point cannot represent 0.1 exactly — it stores an approximation. The error is tiny, it is always there, and it surfaces the moment numbers are added.
`0.1::numeric`	The same literal cast to `numeric`, which stores decimal digits exactly.
`+ 0.2::numeric;`	Returns exactly `0.3`.

The rule they should be able to recite

Any column holding money — fees, balances, amounts — is `numeric`, never `float` or `double`. This is a favourite interview question and a real-world bug factory. `numeric(8, 2)` fixes eight total digits with two after the point, which is exactly right for a course fee.

Dates and times: `date`, `timestamp`, `timestamptz`

`date` is a calendar date with no time and no zone: `2026-10-19`. Always write date literals in ISO order, `'2026-10-19'`, never a local format.

For "when did this happen" there are two timestamp types, and the class rule is absolute: **use `timestamptz`**. It stores the true instant, normalised to UTC internally, and displays it in whichever time zone the current session is set to. The without-zone version records a bare `10:30` with no idea *which* 10:30 — a bug that appears only when one person's laptop and a server elsewhere disagree.

`now()` returns the current instant, evaluated when the row is created. It is what `created_at` uses.

Booleans

`boolean` holds `true` or `false`. PostgreSQL also accepts `'t'`, `'yes'` and `1`, which is precisely why the class writes only `true` and `false`. A later module adds an `active` flag to enrollments.

The decision table

Work through this row by row and have students justify each one aloud before moving on.

Column	Type	Why
`students.full_name`	`text NOT NULL`	Any length, always present
`students.email`	`text NOT NULL`	Uniqueness arrives with Module 3's constraints
`students.date_of_birth`	`date`	Store the date, compute the age — an age column goes stale every birthday
`courses.course_code`	`text NOT NULL`	A code like `CS101` contains letters and leading zeros a number would destroy
`courses.title`	`text NOT NULL`	Text, unbounded
`courses.credits`	`integer NOT NULL`	A small exact count the business must supply
`courses.course_fee`	`numeric(8,2) NOT NULL`	Money — exact decimal, never float
`*.created_at`	`timestamptz NOT NULL DEFAULT now()`	The only value the server should invent for you

Two design habits, named explicitly

Store the fact, compute the derived value

A student's *age* changes every single day. Their *date of birth* never does. Storing the age means every row silently becomes wrong on every birthday, and there is no way to reconstruct the truth later. Derived columns are a maintenance trap students should learn to spot in code review.

A default is a claim, not a convenience

Use a `DEFAULT` only when one value is genuinely the norm. `course_fee numeric(8,2) NOT NULL` with no default means a missing fee is an error the student must supply. `DEFAULT 0` would silently invent a free course that nobody approved — a data-quality bug that is invisible until someone balances a ledger. This is why the only `DEFAULT` in this schema is `now()`.

Building the second table

With the class calling out each type and default *before* it is typed:

Listing 10. Creating the courses table

```
CREATE TABLE courses (
    id          integer GENERATED BY DEFAULT AS IDENTITY,
    course_code  text NOT NULL,
    title        text NOT NULL,
    credits      integer NOT NULL,
    course_fee   numeric(8, 2) NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);
```

Line by line — only what differs from `students`

Line	How it differs, and why
`course_code text NOT NULL`	The short public code, such as `CS101`. Text, not integer — it contains letters and leading zeros that a numeric type would destroy.
`title text NOT NULL`	The full course name. Text, unbounded.
`credits integer NOT NULL`	A small exact count, with **no default**. `DEFAULT 1` would invent a value the registrar may never have approved.
`course_fee numeric(8, 2) NOT NULL`	The price, exact to two decimal places. Money, so `numeric`; and **no default**, so a missing fee cannot silently become zero.
`id, created_at, PRIMARY KEY`	Identical in shape to `students`. Consistency across tables is what makes a schema readable six months later.

Point at the hole now, while it is fresh. `\dt` shows `students` and `courses`, but the project needs `enrollments`, and it cannot be built properly yet — it has to *reference* rows in both tables above, which is precisely what the next module's constraints are for. Keep the gap visible and let it motivate the next session.

Auto-generated identifiers and UUIDs

The problem first

In the Python stage, students assigned identifiers themselves: `id = len(students) + 1`, or a shared counter variable. Ask what happens when two API processes handle two requests at the same moment. Both compute `id = 5`. Both write. The dictionary never noticed.

A real database serving real traffic notices immediately. Identifier assignment is the server's job, and the moment you accept that, the syntax stops being something to memorise and becomes something to reason about.

The old spelling and the standard one

Listing 11. serial and identity, side by side

```
-- Old, PostgreSQL-specific (you will meet it everywhere online):
id  serial PRIMARY KEY

-- Standard SQL, and what this course uses:
id  integer GENERATED BY DEFAULT AS IDENTITY
```

Line by line

Part	What it does, and why
`serial`	Creates a hidden counter (a *sequence*) and sets the column's default to draw from it. It works, but it is a PostgreSQL invention rather than standard SQL, and the column itself is not truly protected from manual interference.
`PRIMARY KEY`	Attached directly, because `serial` is a bare type shorthand.
`integer`	The actual column type, stated explicitly — here the type is the SQL standard's `INTEGER`.
`GENERATED BY DEFAULT`	Server-generated, but explicit values are still permitted. This is what makes seeding data and later migrations possible.
`AS IDENTITY`	Standard SQL syntax for "the server owns the counter".
`GENERATED ALWAYS`	The stricter alternative: it refuses explicit values unless a statement writes `OVERRIDING SYSTEM VALUE`. Worth knowing; not needed in this course.

One observable difference worth naming, even without a demonstration: in `\d` output an identity column reads `generated by default as identity`, whereas a `serial` column shows a visible default of `nextval('students_id_seq'::regclass)`. **When students meet `serial` in the wild, they should now read it as identity — the old spelling.**

Where the counter went

Nothing has been inserted yet, so nothing can be watched counting. But `\d students` shows it: the `id` line ends with `generated by default as identity`. The counter proves itself in Module 4, the first time a row goes in. Let that be a hook rather than a loose end.

UUIDs — the alternative identifier

An identity identifier is small, fast and sequential — which also makes it *guessable*. If `/students/7` exists, then `/students/8` very probably does too. PostgreSQL has a native `uuid` type, and since version 13 a built-in generator requiring no extension at all.

Listing 12. A UUID column with an automatic default

```
id  uuid DEFAULT gen_random_uuid()
```

Line by line

Part	What it does, and why
`uuid`	A built-in type storing a 128-bit identifier, comparable to the standard `uniqueidentifier` type in Microsoft SQL Server.
`DEFAULT`	Fills the column automatically when a row is inserted, exactly as `now()` does for a timestamp.
`gen_random_uuid()`	The built-in generator, available since PostgreSQL 13. Older tutorials insist on `CREATE EXTENSION pgcrypto` first — that advice is obsolete for any supported version and must not be taught.

Random UUIDs are effectively unguessable and collision-proof across machines, which is why distributed systems and public-facing APIs choose them. The costs are real: twice the storage of a `bigint`, and randomness scatters newly inserted rows through the index instead of appending at the end.

Where this course draws the line

Integer identity for internal primary keys; awareness of UUIDs. A later API stage will want public, non-guessable identifiers, and students will meet UUIDs there. Give them the concept now so it is not a surprise, but do not let it swallow this module — the identifier problem is already solved, and solved correctly, by identity columns.

Changing a table: `ALTER TABLE`

Requirements change, and tables must change with them rather than being dropped and rebuilt, because rebuilding loses the data. `ALTER TABLE` is a family of statements; these four are the ones students will actually use.

Listing 13. The four structural changes, and how to undo one of them

```sql
-- Add a column
ALTER TABLE students ADD COLUMN phone text;

-- Rename a column (free, reversible, good practice)
ALTER TABLE students RENAME COLUMN phone TO phone_number;

-- Change a column's type
ALTER TABLE students ALTER COLUMN phone_number TYPE varchar(20);

-- Remove a column - data deleted from every row, no confirmation
ALTER TABLE students DROP COLUMN phone_number;
```

Line by line

Statement	What it does, and why
`ALTER TABLE students`	Every one of these names its target table. `ALTER` means *change the existing structure*, never create a new one.
`ADD COLUMN phone text`	Adds a nullable `text` column. Nullable by default, so existing rows are unaffected. This is instant even on very large tables — and since PostgreSQL 11, adding a column *with* a default is instant too, because the server records the default rather than rewriting every row.
`RENAME COLUMN phone TO phone_number`	Renames without touching any data. The safest structural change there is: nothing is lost and `\d` confirms the new name immediately.
`ALTER COLUMN ... TYPE varchar(20)`	Reinterprets the column as `varchar(20)`, converting existing values. The server must convert every existing value, so this can *fail* if the data does not fit — text containing letters cannot become `integer`.
`DROP COLUMN phone_number`	Deletes the column and its data from every row. **The data is gone, immediately, with no confirmation prompt.** `ALTER TABLE` never asks whether you are sure; the database assumes the person typing is sure. This is why the habit in the next section exists.

One further form is worth recognising without memorising, for the day a conversion genuinely needs help:

Listing 14. An explicit conversion when PostgreSQL would otherwise guess

```
ALTER TABLE courses
    ALTER COLUMN credits TYPE numeric USING credits::numeric;
```

Line by line

Part	What it does, and why
`TYPE numeric`	The target type. Widening is safe; narrowing — `numeric` to `integer`, say — silently rounds, which is its own lesson.
`USING credits::numeric`	The explicit conversion expression PostgreSQL should apply to each existing value. Without `USING`, the server picks a conversion that may not be the one intended.

The lesson to land

`ALTER TABLE` is powerful, silent, and unforgiving. Every change gets verified with `\d`, and every risky change gets rehearsed before it is committed — which is exactly what the next section provides.

Removing a table: `DROP TABLE`, and the safety net

Listing 15. Two ways to drop a table

```
DROP TABLE students;              -- removes the structure and every row, permanently
```

```
DROP TABLE IF EXISTS students;    -- script-safe: drops it if present, silent if not
```

Line by line

Statement	What it does, and why
`DROP TABLE students;`	Removes the table definition *and every row in it*, at once. One line of SQL, no confirmation, no recycle bin. Recovery means a backup.
`DROP TABLE IF EXISTS students;`	The same, except that a missing table is not an error. This is precisely what makes a setup script re-runnable: it succeeds against a fresh database and against an existing one.

Now the demonstration that makes PostgreSQL memorable. **SQL changes in PostgreSQL run inside transactions** — the same `BEGIN`, `COMMIT` and `ROLLBACK` that protect data protect *structure* as well.

Listing 16. Rehearsing a destructive change, then undoing it

```
school_db=# BEGIN;
BEGIN
school_db=# DROP TABLE students;
DROP TABLE
school_db=# \dt
        (students is gone - only courses remains)
school_db=# ROLLBACK;
ROLLBACK
school_db=# \dt
        (students is back, untouched)
```

Line by line

Line	What it does, and why
`BEGIN;`	Opens a transaction — a block of work that succeeds or fails as a unit. `psql` normally *autocommits*: every statement commits the instant it succeeds. `BEGIN` suspends that behaviour.
`DROP TABLE students;`	The drop really does happen, inside the open transaction. Nothing is committed yet.
`\dt`	Lists tables. `students` is missing — the fear is real, and entirely deserved.
`ROLLBACK;`	Discards the entire transaction, undoing everything since `BEGIN`.
`\dt`	`students` is back, with its structure and every row. The safety net, demonstrated rather than promised.

The professional habit to drill: **rehearse a dangerous statement inside a transaction.** `BEGIN`, run the drop, inspect with `\dt` and `\d`, then choose `COMMIT` for real or `ROLLBACK` for never-mind.

Say the limit plainly, because the alternative is a student who learns the wrong lesson. **The net exists only until you commit.** Once the transaction is committed — or once the statement is run outside `BEGIN`, where `psql` autocommits — a dropped

table is gone, and recovery means a backup.

One sentence of contrast for later: `TRUNCATE TABLE students` deletes all the data but keeps the structure. Its full story belongs beside `DELETE` in Module 4, where the difference finally matters.

The script file, and proving it is re-runnable

Students keep every `CREATE` and `ALTER` in `db/sql/01_create_tables.sql`, numbered so the order is never guessed. This is not tidiness. It is the seed of what real teams call *migrations*, which is Day 3 conceptual material, and it is the artefact students will still have at the end of the course.

Listing 17. The complete 01_create_tables.sql

```
-- ============================================================
-- 01_create_tables.sql
-- Project : Student Enrollment API
-- Module  : Day 2 - Module 2 (Creating and defining structures)
-- Target  : PostgreSQL 18, database school_db, role student
--
-- How to run:
--   terminal   : psql -U student -d school_db -f db/sql/01_create_tables.sql
--   inside psql: \i db/sql/01_create_tables.sql
--   VS Code    : open this file and press F5
--
-- This file is re-runnable. Running it twice is safe, and is
-- the correct way to rebuild the schema from zero.
-- ============================================================


-- ------------------------------------------------------------
-- Section 1 - Remove any previous version of these tables.
-- Dependents first: nothing depends on anything yet, but the
-- habit starts now. When enrollments arrives with its
-- references, dropping it before its parents is required.
-- ------------------------------------------------------------
DROP TABLE IF EXISTS enrollments;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS students;


-- ------------------------------------------------------------
-- Section 2 - students
-- One record per person on the register.
-- ------------------------------------------------------------
CREATE TABLE students (
    id              integer GENERATED BY DEFAULT AS IDENTITY,
    full_name       text NOT NULL,
    email           text NOT NULL,
    date_of_birth   date,
    created_at      timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);
```

```
-- ------------------------------------------------------------
-- Section 3 - courses
-- One record per course the school offers.
-- ------------------------------------------------------------
CREATE TABLE courses (
    id           integer GENERATED BY DEFAULT AS IDENTITY,
    course_code  text NOT NULL,
    title        text NOT NULL,
    credits      integer NOT NULL,
    course_fee   numeric(8, 2) NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (id)
);


-- ------------------------------------------------------------
-- Section 4 - enrollments
-- DELIBERATELY NOT CREATED YET.
--
-- This table must REFERENCE rows in students and courses. That
-- requires FOREIGN KEY, which is Module 3. Creating it now
-- would produce a table that can point at students who do not
-- exist - the exact bug that constraints exist to prevent.
-- ============================================================
```

Line by line

Line or block	What it does, and why
`The -- comment lines`	A `--` comment runs to the end of the line. Executed by nobody, read by everybody — a file that explains its own run commands is self-documenting.
`The How to run block`	Records all three ways of executing the file, so the next reader — or next term's student — is never left to guess.
`The ``re-runnable'' note`	States the design intent before any code. Re-runnability is the reason the drops come first; it is a goal, not an accident.
`DROP TABLE IF EXISTS enrollments;`	Drops the join table *if it happens to exist*. Today it does not, so this does nothing and reports no error. Naming the future table here keeps the drop order correct from day one.
`... courses; then ... students;`	Removes the current schema in dependency order. When `enrollments` exists it references these two, so the child must be dropped first or the statement fails with "other objects depend on it".
`The two CREATE TABLE blocks`	The module's actual deliverable, identical to Sections 3 and 4, now living in a file instead of being typed by hand each time.
`The Section 4 comment`	Documents the missing table and, crucially, *why* it is missing. Students see the gap as deliberate rather than forgotten, and carry the question into Module 3.

Running it — three routes, one file

Listing 18. Run the script from a terminal, from inside psql, or from VS Code

```
# Terminal, from the project folder
psql -U student -d school_db -f db/sql/01_create_tables.sql
```

Listing 19. The same file, run from inside an open psql session

```
\i db/sql/01_create_tables.sql
```

Listing 20. The VS Code route

```
Open db/sql/01_create_tables.sql in the editor, confirm that
school_db is the active connection in the PostgreSQL panel,
then press F5 (or the play button the extension adds).
```

Line by line

Part	What it does, and why
`psql`	Opens the PostgreSQL client — the same program Module 1 installed and verified.
`-U student`	Connects as the `student` role. A password prompt follows. **PowerShell and Windows terminals display nothing while the password is typed** — not even asterisks. That is normal, and it unsettles almost every student the first time.
`-d school_db`	Connects straight to `school_db`. Without it, `psql` opens a database named after the user, and every later command would run in the wrong place.
`-f`	Executes the file, then exits.
`The path after -f`	Relative to the **current directory**, so the command must be run from the project folder.
`\i`	"Include": runs a file from inside an open session.
`The path after \i`	Relative to **psql's** working directory, *not* VS Code's. A `No such file or directory` error almost always means the terminal is in the wrong folder.
`F5 in VS Code`	Runs the open file against the selected connection. Convenient — but watch the output panel, because a silent run is a failed run.

Prove it is re-runnable

Run the file a second time, immediately after the first. It must succeed with no errors at all, because the `DROP ... IF EXISTS` lines cleared the way. Then confirm with `\dt` and `\d students`.

The single most common failure of this section

If the second run reports `relation "students" already exists`, the drop lines were missing or mistyped. This is worth catching deliberately: a script that

> only works once is not a script, it is a transcript of what you happened to type.

Verification checklist

Students verify their own work with the two tools they already own, `\dt` and `\d`.

#	Check	Command	Passes when
1	Tables exist	`\dt`	`students` and `courses` are both listed
2	Columns are right	`\d students`	Five columns, with exactly the agreed names
3	Types are deliberate	`\d students`	`integer`, `text`, `date` and `timestamptz` all visible — no accidental text-everything
4	Identity works	`\d students`	The `id` line ends `generated by default as identity`
5	Defaults are only `now()`	`\d courses`	`created_at` defaults to `now()`; `credits` and `course_fee` show **no default**
6	Alter round-trip	add, `\d`, drop, `\d`	The column appears, then disappears, and nothing else changed
7	Rename round-trip	`RENAME COLUMN`, `\d`	The new name is shown and no duplicate remains
8	Transactional drop rehearsed	`BEGIN`, drop, `\dt`, `ROLLBACK`, `\dt`	The table vanishes, then returns
9	Script is re-runnable	run the script twice	The second run succeeds with no error
10	Project files in place	open the folder in VS Code	The script, `.gitignore`, `.env.example` and `README.md` are all present

Troubleshooting

Nearly every problem in this module falls into this table. Teach students to read the error rather than to search for it, because searching an error message leads to forum posts describing a different system, a different version, or a different operating system.

Error from psql	Likely cause	Fix
`syntax error at or near "("`	A comma is missing between two column definitions	Look at the line *above* the one named in the error — that is where the comma belongs
`relation "students" already exists`	The table was created earlier in this session	`\dt` to confirm, then drop it deliberately, or add `IF EXISTS` to the script
`relation "students" does not exist`	A typo, or the wrong database	`\conninfo`, `\c school_db`, then check the spelling
`column "fullname" of relation ... does not exist`	A column-name typo inside an `ALTER`	`\d students` and copy the exact name
`type "timestampz" does not exist`	A misspelled type — it is `timestamptz`	Check against the type table in Section 4
`unterminated quoted string`	A single quote was never closed	Find the odd quote; press `Ctrl+C` to cancel and retype the statement
`must be owner of table students`	The table was created by a different role, such as `postgres`	Reconnect as the owning role, or ask the owner to drop it
`No such file or directory on \i or -f`	The terminal is in the wrong directory	`cd` into the project folder first — paths are relative to where the command runs, not to VS Code
`Prompt shows school_db-# and nothing happens`	The statement is missing its semicolon	Type `;` and press Enter
`Second run of the script fails`	The `DROP ... IF EXISTS` lines are missing or mistyped	Restore the three drop statements to the top of the file

Instructor notes

Time. Roughly 40 minutes of teaching and 35 minutes at the machine, inside the Day 2 session. Section 4, data types, is the time sink by design — it *is* the module. If you are running late, compress the folder setup; never compress the data types.

Misconceptions to catch head-on. That `varchar(n)` is required or faster — in PostgreSQL it is neither. That `serial` and identity are the same thing. That `DROP` can be undone — only *uncommitted* drops can. That `float` is acceptable for money. And the one this revision corrects: that a `DEFAULT` is a convenience. A default on a price column invents a price nobody agreed to.

The float demonstration. Run `0.1::float8 + 0.2::float8` live, twice, slowly. It is the most memorable two minutes in SQL teaching and it will survive long after the syntax is forgotten.

The gasp moment. The `BEGIN`, drop, `\dt`, `ROLLBACK` sequence reliably produces the best reaction of the session. Let a student run it at their own machine rather than watching you do it. It also quietly pre-tears the seam for Day 3's transactions module — say "remember this on Day 3" and move on.

Assessment cues. Ask why a student's age should not be stored — expect *store the date of birth, compute the age, it goes stale*. Ask what type `course_fee`

is and why — expect `numeric` and a sentence about rounding. Ask what `GENERATED BY DEFAULT AS IDENTITY` actually saves students from — expect an answer about duplicate identifiers under concurrency, not "it is automatic". Ask why `course_fee` carries no default — expect that a default would invent business data.

Where this goes next. Module 3 supplies the missing vocabulary — `PRIMARY KEY` properly, `FOREIGN KEY`, `UNIQUE`, `CHECK` — and with it, at last, the `enrollments` table. Open the next session by asking what happens if someone inserts an enrollment for a student identifier that does not exist, and let that question motivate the module.

On the folder setup. Students who have never used VS Code as a project tool will lose ten minutes here. Demonstrate it on the projector *before* the session, showing `File > Open Folder` and the Explorer sidebar, so that the only thing happening during the session is typing SQL.

Module 2 submission

Each student submits the file `db/sql/01_create_tables.sql`; the output of `\d students` and `\d courses` from their own database; evidence that the script ran twice with the second run producing no errors; a transcript of the transactional-drop rehearsal from `BEGIN` through `ROLLBACK`; and three short written answers:

1. Why `timestamptz` rather than `timestamp` for `created_at`?
2. Why identity columns rather than application-assigned identifiers?
3. What happens to existing row data when `DROP COLUMN` runs?

The third answer has a deceptively simple correct response — *it is deleted, and no warning is given* — and it is worth grading carefully, because a student who misses it has not yet understood what this module is for.

Sources

postgresql.org/docs/18/sql-createtable.html · postgresql.org/docs/18/ddl-alter.html · postgresql.org/docs/18/datatype.html · postgresql.org/docs/18/datatype-numeric.html · postgresql.org/docs/18/datatype-datetime.html · postgresql.org/docs/18/datatype-uuid.html · postgresql.org/docs/18/sql-droptable.html · postgresql.org/docs/18/sql-truncate.html · postgresql.org/docs/18/functions-datetime.html · postgresql.org/docs/18/tutorial-transactions.html · postgresql.org/docs/18/app-psql.html · postgresql.org/docs/18/sql-altertable.html